

17 | Vault（一）：企业级敏感数据的守护者

潘野 · 云原生基础架构实战课



你好，我是潘野。

在上一讲里，我们已经深入了解了云原生的安全体系。从今天开始，我们继续学习如何在云环境中真正落地这样的安全体系。

假设你是公司的信息安全工程师，想象一下，我们的业务开发过程中有好几种环境。有的是纯粹的开发测试环境，有的是上线前的预发布环境。每个环境都有自己的一套数据库密码、API 密钥和 TLS 证书。

那么问题来了，如何确保这些敏感信息，只在正确的环境中被正确的人使用呢？今天这一讲，我们就从这个问题聊起。

应对策略分析

在云环境中管理不同环境的敏感信息是一个重要的安全挑战。我们需要一个安全、灵活且可扩展的解决方案来妥善保护和分发这些敏感数据。总的来说，应对这一挑战的策略可以概括为这样几点。

1. 先分个类，看看风险

这个步骤很关键，我们需要把所有的敏感信息都梳理一遍。然后根据它们的重要性和风险等级进行分类。这样一来，我们就可以明确知道哪些数据是需要特别保护的，比如涉及财务信息、个人隐私数据或者业务关键的机密信息。

2. 加密！加密！还是加密！

加密是保护我们数据安全的关键武器，尤其是在云环境中。而加密具体可以从这三个方面入手。

第一，在传输中加密。确保所有的数据在网络中传输时都是加密的，这是基本操作。无论是网站数据还是内部系统之间的数据交换，都应该使用 TLS 来加密通信。

第二，静态数据也要加密。数据在存储时同样需要保护。使用强加密标准（比如 AES-256），可以确保数据在数据库、硬盘或任何存储介质上都不会轻易被破解。这种加密可以有效抵御未经授权访问和数据泄露的风险。

第三，管理好你的密钥。密钥是开启加密数据的钥匙，所以保护好密钥就等于保护了数据。使用专门的密钥管理系统，如 AWS KMS（亚马逊密钥管理服务）或 HashiCorp Vault，可以帮助我们安全地存储、管理和轮换密钥。这些系统还提供了密钥的使用记录和审计功能，帮助我们满足合规要求并及时发现任何异常使用密钥的行为。

3. 谁能看，谁不能看

完成了前两步，还得按“角色”来设置权限。

首先我们要通过定义角色，也就是 RBAC 的方式来控制对资源的访问权限。权限不是直接分配给个别用户，而是分配给不同角色，然后再给不同的用户分配不同角色。例如，你可以创建

一个“数据库管理员”角色，赋予它访问和修改数据库的权限；而一个“普通员工”角色可能只有查看数据库中某些数据的权限。

除了访问控制本身，详细的审计日志也是安全管理中不可或缺的一环。我们应该记录每一次对敏感信息的访问请求，包括成功和失败的尝试。这不仅有助于安全审计，也有助于合规性检查和监测潜在的安全威胁。

方案需求

所以根据上面的策略需要，我们需要一个系统能够做到以下几点：

能够集中管理所有敏感数据

采用强大的加密算法对敏感数据进行加密，即使数据被泄露，也无法被破解。

采用 RBAC 方式对用户访问敏感数据的权限进行细粒度控制

方便用户访问敏感数据，提供统一的访问接口，例如 Web 界面、命令行界面、API 接口等，方便用户或者程序能访问敏感数据。

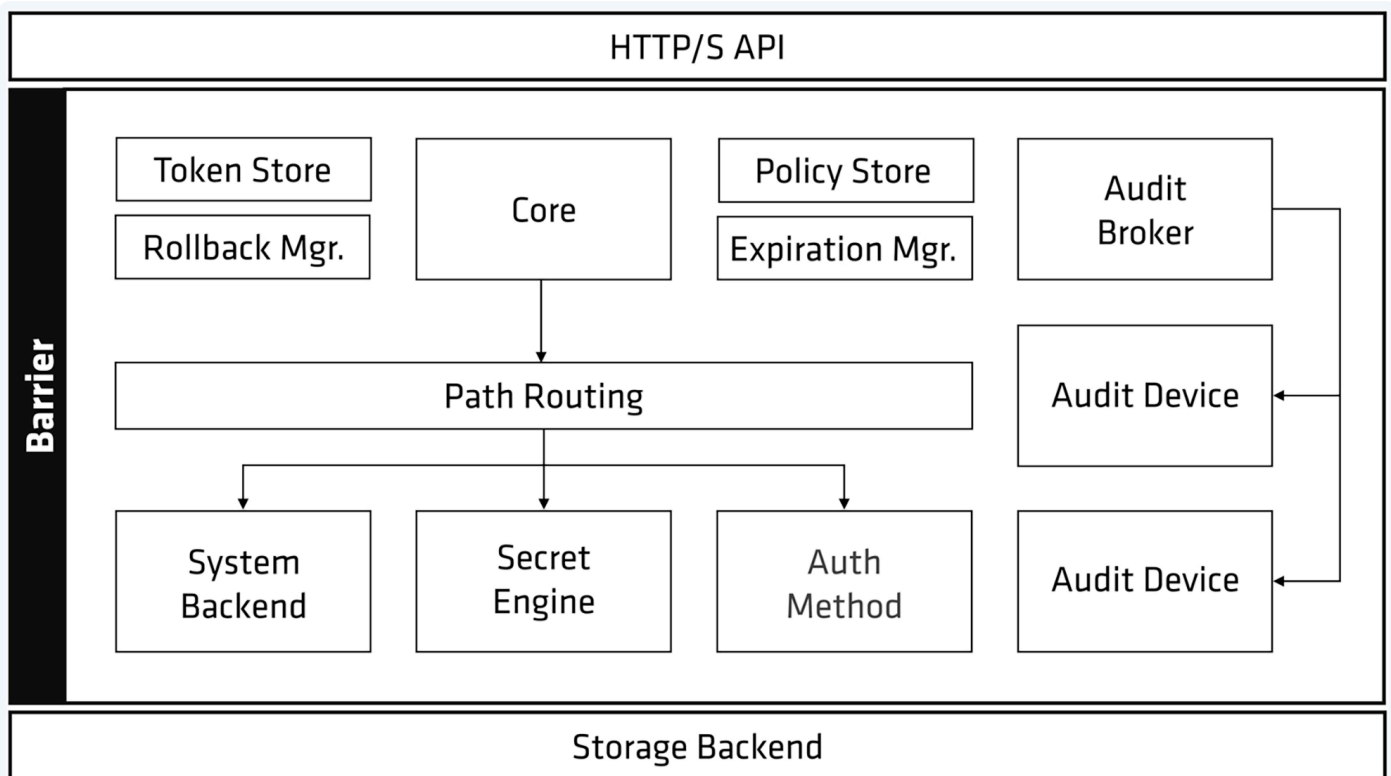
在业界，通常有这么几种解决方案。

第一种方案，云服务提供商通常提供密钥管理服务（如 AWS KMS、Azure Key Vault、Google Cloud KMS）。这些服务结合云服务提供商的身份和访问管理（IAM）工具，可以实现细粒度的访问控制，帮助用户安全地生成、存储和管理加密密钥。

第二种就是开源解决方案 Vault，Vault 是由 HashiCorp 开发的一个开源工具，专门用于管理敏感数据，如 API 密钥、密码和证书等等。Vault 的功能非常强大，同时适用于云上环境与云下环境，可以说是企业级加密方案最佳选择之一。

Vault 架构和核心概念

为了帮助刚刚接触 Vault 的同学加深了解，我们先看看 Vault 的大体框架。

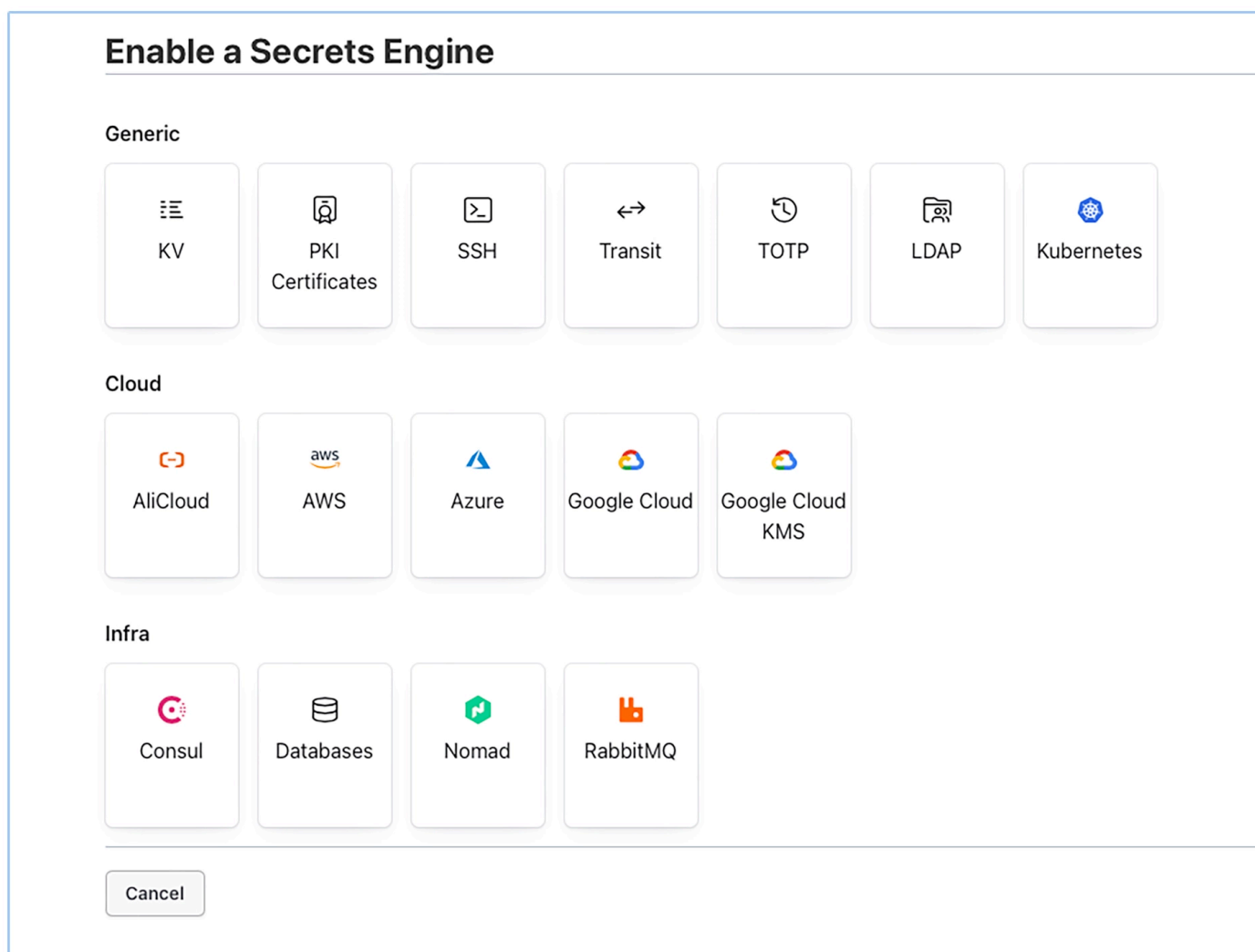


图片来自Vault官网: <https://developer.hashicorp.com/vault/docs/internals/architecture>

结合上面这张图，可以发现 Vault 的架构复杂度还是比较高的。不过听完后面的分析，理解起来也并不复杂。

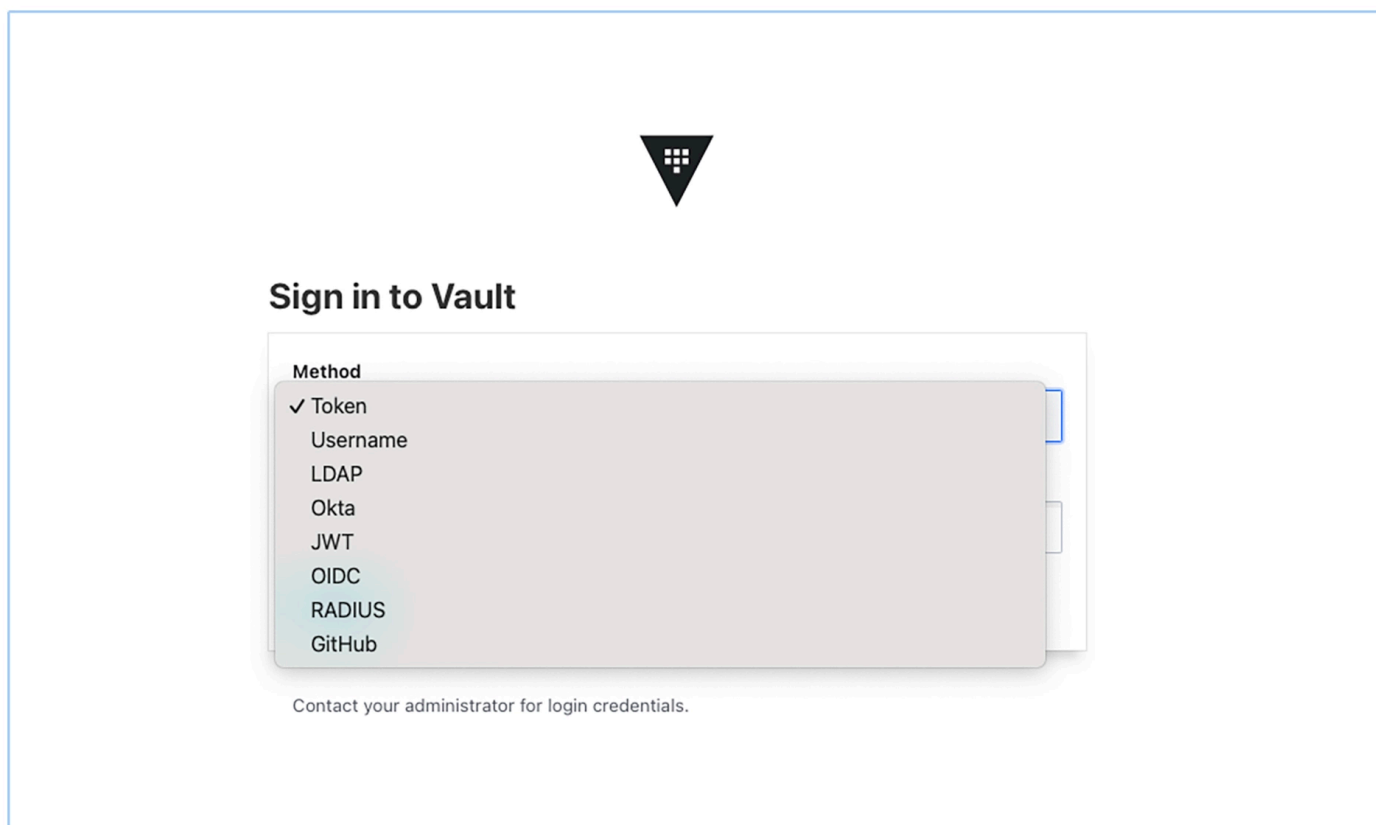
首先，Vault 提供了一个 REST API，用于管理和访问存储在 Vault 中的密钥。API 支持认证授权、读取写入密钥、审计等功能。

其次是 Secrets Engine，这一组件是专门设计的，用来存储、生成和管理访问 Secrets。这些 Secrets 可能包括密码、API 密钥、证书或其他形式的敏感数据。每种类型的密钥引擎都提供了特定的功能来处理不同种类的密钥数据，这让 Vault 能够支持广泛的用例和应用场景。下图是 Vault Secrets Engine 中默认支持的场景。



接下来是 Vault 的存储后端（Storage Backend），它是用来持久化存储其内部数据的组件。这些数据包括密钥、令牌、策略和其他各种配置信息。选择合适的存储后端对于确保 Vault 的性能和可靠性至关重要。常见的 Vault 后端存储有 Consul、PostgreSQL、Filesystem 以及 S3 对象存储等。

然后我们再来看看 Vault 的认证方法（Auth Methods），它是用于验证和授权用户或系统访问 Vault 存储的密钥机制。Vault 支持多种认证方法，允许用户根据其环境和需求选择合适的认证方式。下图展示了一些常用的 Vault 认证方法，比如 Token、Username 和 JWT 等等。



其他还有审计、规则等等内容，我就不一一展开了。接下来，我们直接进入实践环节，看看如何使用 Vault 来管理我们的密钥。

用 Vault 管理应用密钥

现在假设我们向 Kubernetes 集群上部署了一个用 Python Django 写的 Web 应用，这个 Web 应用需要去连接一个数据库。一般来说，我们需要在 Django 的 `settings.py` 中配置数据库的信息，大致格式如下：

```
1 DATABASES = {  
2     'default': {  
3         'ENGINE': 'django.db.backends.mysql',  
4         'NAME': 'your_database_name',  
5         'USER': 'your_database_user',  
6         'PASSWORD': 'your_database_password',  
7         'HOST': 'localhost', # 或者使用具体的 IP 地址  
8         'PORT': '3306',      # 默认 MySQL 端口为 3306  
9     }  
10 }
```

一般在 Kubernetes 中，通用做法是将数据库的 Host、User 和 Password 等信息存在 Kubernetes Secrets 中，虽然 Kubernetes Secrets 提供了一种存储敏感信息的方式，但它们本身并不提供真正的“加密”机制。Secrets 通常是以 base64 编码存储的，这只是一种编码形式，而非加密。任何拥有适当权限的用户都可以轻松地解码 base64 数据。

因此，仅仅使用 Kubernetes Secrets 并不能保证数据的安全性。所以，我们还要利用 Vault 管理 Kubernetes Secrets，才能有效地提升系统和应用的安全性。

接下来，我们逐一过一下具体的部署和操作过程。Vault 有很多种安装方式，可以选择安装在 Linux 或者 Mac 系统中。这里推荐你安装在 Kubernetes 集群中，这里我们采用 helm 方式部署。

配置 Vault Helm chart

我们执行如下命令，在本地添加 vault 的 helm 仓库，并且对 vault 的 chart 做一些定制。

```
1 helm repo add hashicorp https://helm.releases.hashicorp.com  
2 helm repo update
```

然后，我们来创建一个名为 `vault-values.yaml` 的文件，以便自定义 Vault 的安装。这里是一个基本的配置示例。对于刚接触 Vault 的同学，可以使用 standalone 的模式，同时将

Vault 的 UI 打开，方便学习和理解。如果是在生产环境下部署，请打开 ha 模式，保证 Vault 的高可用。

 复制代码

```
1 server:
2   standalone:
3     enabled: true
4   ha:
5     enabled: false
6   dataStorage:
7     enabled: true
8     size: 10Gi
9   service:
10    type: LoadBalancer
11 ui:
12   enabled: true
```

安装 Vault

接下来，需要使用 Helm 和你的配置文件来把 Vault 部署到 Kubernetes 集群上，并检查部署状态，确保 Vault 的 pod 启动起来。

 复制代码

```
1 helm install vault hashicorp/vault -f vault-values.yaml
```

初始化和解封 Vault

Vault 安装后还需要初始化和解封。我们需要运行以下命令来初始化 Vault，这将输出一些非常重要的信息，包括解封密钥和初始根令牌，请确保安全地保存这些信息。

 复制代码

```
1 shell> kubectl exec -it vault-0 -- vault operator init
2
3 Unseal Key 1: Yg8rQm...uzzdwf
4 Unseal Key 2: t3TgA5...vDE2
5 Unseal Key 3: Lrj....0hMLgo1
```

```
6 Unseal Key 4: 38NKd....SSe+h9
7 Unseal Key 5: ue6XR....B02ca/o
8
9 Initial Root Token: hvs.x2Vwusl..CNlpD
10
11 Vault initialized with 5 key shares and a key threshold of 3. Please securely
12 distribute the key shares printed above. When the Vault is re-sealed,
13 restarted, or stopped, you must supply at least 3 of these keys to unseal it
14 before it can start servicing requests.
```

请注意，使用解封密钥解封 Vault，一般需要输入三次，用三个不同的 unseal key。

 复制代码

```
1 kubectl exec -it vault-0 -- vault operator unseal <UnsealKey1>
```

如果看到如下状态，就代表我们已经完成了安装。

 复制代码

```
1 ~ >>> kubectl get pod -n vault
2 NAME                                READY   STATUS    RESTARTS   AGE
3 vault-0                             1/1     Running   0           18h
4 vault-agent-injector-d986fcb9b-jwhc5 1/1     Running   0           18h
```

Vault Agent 自动注入

Vault Agent 注入器是一个用于自动化管理和注入 Secrets 到 Kubernetes Pod 中的工具，由 HashiCorp Vault 提供。通过这个功能，Kubernetes 环境中的应用才能安全地获取存储在 Vault 中的敏感信息，而无需在应用代码中硬编码这些信息。

我们这就了解一下 Vault Agent 注入器应该如何应用。

第一步，启用 Kubernetes 认证方法。

首先，你需要在 Vault 中启用 Kubernetes 认证方法。这可以通过 Vault CLI 或 API 完成。使用 CLI 的命令如下：

```
1 vault auth enable kubernetes
```

 复制代码

这个命令将在 Vault 中启用 Kubernetes 认证。

第二步，配置 Kubernetes 认证方法。

这个步骤的目的是让 Vault 可以与 Kubernetes API 服务器通信并验证 Token。这包括设置 API 服务器的地址、服务账户的 JWT Token 以及 Kubernetes 的 CA 证书。

请注意，在配置认证之前，需要先收集相关信息，我们来看看具体如何操作。

获取 Kubernetes 集群的相关信息

1. API Server 地址：通常可以通过这个命令获取。

```
1 kubectl cluster-info | grep 'Kubernetes master' | awk '/http/ {print $NF}'
```

 复制代码

2. Kubernetes CA 证书：CA 证书通常位于

```
/var/run/secrets/kubernetes.io/serviceaccount/ca.crt
```

3. Token Reviewer JWT：创建一个服务账户，或者使用现有的服务账户，并获取其 Token。
例如，创建一个名为 `vault-auth` 的服务账户，代码就是后面这样。

```
1 kubectl create serviceaccount vault-auth
2 kubectl create clusterrolebinding vault-auth-delegator --clusterrole=system:auth-
```

 复制代码

之后还需要获取 Token。

 复制代码

```
1 SECRET_NAME=$(kubectl get serviceaccount vault-auth -o jsonpath="{.secrets[0].name}")
2 TR_JWT=$(kubectl get secret $SECRET_NAME -o jsonpath="{.data.token}" | base64 --d
```

然后，我们就可以使用前面收集的信息，配置 Vault 的 Kubernetes 认证了。

 复制代码

```
1 vault write auth/kubernetes/config \
2     token_reviewer_jwt="$TR_JWT" \
3     kubernetes_host="<KUBERNETES_API_SERVER_ADDRESS>" \
4     kubernetes_ca_cert="@/path/to/ca.crt"
```

第三步，创建角色。你需要在 Vault 中创建一个或多个角色，这些角色定义了哪些 Kubernetes 服务账户可以在哪些命名空间中访问哪些 Vault 资源。

例如，我们创建一个角色，允许名为 `myapp` 的服务账户在 `default` 命名空间中进行访问。

 复制代码

```
1 vault write auth/kubernetes/role/myapp \
2     bound_service_account_names=myapp-sa \
3     bound_service_account_namespaces=default \
4     policies=my-policy \
5     ttl=1h
```

这里 `my-policy` 是我们预先定义的 Vault 策略。

配置完这些步骤后，你就可以使用 Kubernetes 的服务账户 Token 来认证 Vault，并根据定义的角色和策略访问 Vault 中的资源了。

测试

在完成上述配置之后，你可以通过在 Kubernetes Pod 的定义中添加适当的注解，利用 Vault Agent 注入器自动将所需的密钥注入到你的应用中。

这里是一个示例 Pod 定义，展示了如何使用注解。

 复制代码

```
1
2 apiVersion: v1
3 kind: Pod
4 metadata:
5   name: myapp
6   annotations:
7     vault.hashicorp.com/agent-inject: "true"
8     vault.hashicorp.com/role: "my-role"
9     vault.hashicorp.com/agent-inject-secret-credentials.txt: "secret/data/myapp/c
10    vault.hashicorp.com/agent-inject-template-credentials.txt: |
11      {{- with secret "secret/data/myapp/config" -}}
12      DATABASE_URL={{ .Data.data.url }}
13      DATABASE_PASSWORD={{ .Data.data.password }}
14      {{- end }}
15 spec:
16   containers:
17     - name: myapp
18       image: myapp:latest
```

在这个示例中，有几个重要参数需要你理解含义：

`vault.hashicorp.com/agent-inject: "true"` 表示启用自动注入。

`vault.hashicorp.com/role: "my-role"` 指定用于登录到 Vault 的角色。

`vault.hashicorp.com/agent-inject-secret-credentials.txt` 指定从 Vault 获取的密钥路径。

`vault.hashicorp.com/agent-inject-template-credentials.txt` 定义了一个模板，用于格式化注入的密钥。

你可以通过查看 Pod 的日志或进入容器内部，通过检查环境变量或文件来进行验证。

额外的一些建议

首先，为了最大限度地提升 Vault 的安全性，建议实施多因素认证（MFA）。MFA 能够为访问控制提供额外的安全层。Vault 支持 [多种 MFA 方法](#)，例如 PingID、TOTP 和 Okta 等等方式

其次，为了确保数据的持久性和在灾难恢复情况下的可用性，定期对 Vault 数据进行备份至关重要。Vault 提供了灵活的备份选项，例如本地文件系统备份，可以将 Vault 数据定期导出到安全的本地存储中。Vault 也可以支持远程存储备份，利用云存储服务如 AWS S3、Google Cloud Storage 或 Azure Blob Storage，可以自动化备份流程并提高数据的地理冗余。

最后还有一点需要注意的是，我们不仅要定期备份，还要定期测试恢复过程，确保在需要时可以迅速且准确地恢复数据。

总结

今天，我们探讨了在企业内部解决数据加密问题的各种解决方案，云服务提供商的数据加密方案通常与其他云服务高度整合，使用方便，但可能导致对特定云平台的依赖。

像 Vault 这类开源解决方案则提供极高的灵活性和可配置性，适合技术实力较强的团队，但维护需求可能较大。而企业级数据保护软件虽功能全面，但相对成本也较高。

我们重点介绍了 Vault 的架构和核心概念，比如数据存储、引擎和策略等，还了解了 Vault 的 **特点**，包括集中式管理、高安全性、易用性。接着，我们通过在 Kubernetes 中从 Vault 获取应用密钥，深入了解了利用 Vault 管理敏感数据的具体操作过程。

总之，保护敏感数据是每个企业都面临的挑战。选择正确的数据加密和访问控制解决方案，对于确保数据安全、遵守法规、维护企业声誉至关重要。

思考题

课程里举了两个场景，你还知道哪些场景适合使用 vault？

欢迎在评论区与我讨论。如果这一讲对你有启发，也欢迎分享给身边更多朋友。

1. 在云环境中管理敏感信息是重要的安全挑战，需要安全、灵活、可扩展的解决方案。
2. 加密在云环境中是保护数据安全的关键，包括传输中加密、静态数据加密和密钥管理。
3. 角色权限设置和审计日志记录是安全管理中不可或缺的一环，可以有效控制访问权限并监测安全威胁。
4. 针对敏感数据管理的需求，需要一个系统能够集中管理所有敏感数据、采用强大的加密算法对敏感数据进行加密、采用 RBAC 方式对用户访问敏感数据的权限进行细粒度控制。
5. 云服务提供商提供的密钥管理服务 and 开源解决方案Vault是业界常见的解决方案。
6. Vault 的架构包括 REST API、Secrets Engine、存储后端、认证方法等组件，提供安全的秘密存储和访问管理。
7. Vault 的认证方法支持多种方式，允许用户根据其环境和需求选择合适的认证方式。
8. Vault 的存储后端对于确保 Vault 的性能和可靠性至关重要，常见的Vault后端存储有Consul、PostgreSQL、Filesystem以及S3对象存储等。
9. Vault 提供了一个 REST API，用于管理和访问存储在 Vault 中的秘密，API 支持认证授权、读取写入密钥、审计等功能。
10. Vault 的Secrets Engine是专门设计的，用来存储、生成和管理访问Secrets，每种类型的秘密引擎都提供了特定的功能来处理不同种类的秘密数据。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (1)

最新 精选



陈斯佳

2024-06-18 来自加拿大

想问一下老师 Vault生成后的密钥和root token 您是怎么保管的？这些我记得第一次初始化后如果丢失是不可复原的吧



1